

Trabajo Práctico 1

Regresión e Introducción a la evaluación de modelos

72.75 — Aprendizaje Automático (Machine Learning)
Instituto Tecnológico de Buenos Aires

2026 Q2 · Defensa: 26/08/2026

Resumen

Se implementan modelos de regresión lineal y polinómica para predecir el costo anual de gastos médicos (**charges**) del dataset *Insurance Charges*, evaluados con validación cruzada de $k = 5$ folds. Todo el aparato de evaluación — partición train/test, k -fold, codificación *one-hot*, estandarizado, ecuación normal, expansión polinómica y Lasso por descenso por coordenadas — está implementado desde cero sobre `numpy`, sin `scikit-learn`, tal como pide el punto 2.2 del enunciado.

El modelo elegido para producción es un **Lasso de grado 2** con $\lambda = 286,37$, que conserva 10 de 44 características y alcanza un **RMSE de test de [PENDIENTE] dólares**. No es el de menor error de validación: se lo elige porque 8 de las 18 configuraciones evaluadas son estadísticamente indistinguibles entre sí, y frente a un empate se prefiere el modelo más simple.

Índice

1. Introducción teórica: separación train / validación / test	1
2. Punto 1 — Limpieza de datos	2
2.1. Variables categóricas	2
2.2. Datos faltantes	2
2.3. Outliers	2
2.4. Características y escalado	3
3. Puntos 2 y 3 — Implementación	4
3.1. El pipeline, de punta a punta	4
3.2. Cómo se separaron train y test, y por qué así	5
3.3. La regresión polinómica sigue siendo lineal en los parámetros	5
3.4. Regularización L_1	5
4. Punto 4 — Resultados	5
4.1. Dónde está el sobreajuste, y cómo se lo reconoce	6
5. Punto 5 — Las tres respuestas	7
5.1. ¿Qué modelo obtuvo menor error?	7
5.2. ¿Cuál se implementaría en una aplicación real?	8
5.3. ¿Qué RMSE se espera en datos nuevos?	8
5.4. Qué características sobrevivieron a la penalización	9
6. Limitaciones	10
7. Guion de la presentación (10 minutos)	11
A. Reproducibilidad	11

1. Introducción teórica: separación train / validación / test

El objetivo de un modelo de regresión no es ajustar bien los datos que ya vio, sino predecir bien datos que **todavía no vio**. Medir el error sobre el propio conjunto de entrenamiento no sirve para eso: el modelo ajusta sus parámetros exactamente para minimizar ese error, así que un RMSE de train bajo puede significar dos cosas radicalmente distintas — que el modelo capturó el patrón real, o que memorizó el ruido particular de esas 1070 filas. Sin un conjunto separado no hay forma de distinguir una cosa de la otra.

Por eso el trabajo separa tres roles, cada uno con un dato distinto y ninguno intercambiable:

- **Entrenamiento (train).** Ajusta los parámetros del modelo, es decir los coeficientes de la regresión.
- **Validación.** No participa del ajuste; se usa para **elegir entre modelos** — qué grado polinómico, qué valor de λ en Lasso. Es la vara con la que compiten las configuraciones.
- **Test.** Se toca **una única vez**, al final, después de que grado y λ ya están fijados. Estima el error que el modelo va a tener en producción, sobre datos genuinamente nuevos.

La razón por la que el test tiene que quedar intacto hasta el final es sutil pero central: en el momento en que su error se usa para decidir algo — elegir un hiperparámetro, comparar dos arquitecturas, hasta simplemente “mirarlo y probar de nuevo” — deja de ser una medida honesta del error de generalización y pasa a ser, de hecho, un segundo conjunto de validación. El número que reporta ya no dice cuánto va a fallar el modelo en el mundo real, sino cuánto se sobreajustó también al test. En este trabajo el conjunto de test (267 filas) se evaluó una sola vez, después de fijar el modelo de producción por validación cruzada.

¿Por qué validación cruzada y no un único split? Con un solo split, el resultado depende de qué filas cayeron del lado de validación por azar: con 1070 datos de train, una partición particular puede favorecer o perjudicar a un modelo simplemente por cómo cayeron los outliers o los casos raros. La validación cruzada evita depender de una sola tirada. Se parte el train en 5 bloques de aproximadamente 214 filas cada uno y se repite el ciclo ajustar–validar 5 veces, dejando cada vez un bloque distinto afuera del ajuste. Así **cada dato valida exactamente una vez y entrena en las otras cuatro vueltas**, y el promedio de los 5 errores tiene mucha menos varianza que el error de un único split: la señal que importa — ¿esta configuración generaliza mejor que aquella? — deja de estar enmascarada por el ruido de qué le tocó a cada partición. Con un n chico como este esa reducción de varianza es decisiva; es la diferencia entre poder distinguir dos configuraciones o no.

2. Punto 1 — Limpieza de datos

2.1. Variables categóricas

El dataset tiene tres variables categóricas: **sex** (binaria), **smoker** (binaria) y **region** (cuatro categorías: *northeast*, *northwest*, *southeast*, *southwest*). Se codificaron con **one-hot** y no con codificación ordinal porque esta última le impone al modelo un orden y una distancia numérica que no existen en los datos: asignar 0, 1, 2, 3 a las regiones le estaría diciendo que *southwest* está más lejos de *northeast* que de *southeast* en algún sentido matemático, cuando la variable es puramente nominal.

Para **region** se generaron 4 dummies pero se **descartó una columna**, que queda como categoría de referencia absorbida en la ordenada al origen. Es la forma estándar de evitar la *dummy variable trap*: si las 4 dummies estuvieran todas presentes sumarían siempre 1, lo cual las hace linealmente dependientes de la columna constante del modelo y vuelve singular la matriz de diseño, de modo que $X^T X$ no se puede invertir para resolver por mínimos cuadrados.

Para las binarias alcanza con **una sola columna** (`sex=male`, `smoker=yes`) en lugar de dos: la segunda categoría es exactamente el complemento de la primera, así que agregarla sería la misma trampa de colinealidad, sólo que con dos categorías en vez de cuatro.

El codificador resultante produce 8 columnas: `age`, `bmi`, `children`, `sex=male`, `smoker=yes`, `region=northwest`, `region=southeast`, `region=southwest`.

2.2. Datos faltantes

El dataset no tiene ningún valor nulo en las 1338×7 celdas originales. Sí apareció **una fila duplicada exacta** — los índices 195 y 581 son idénticos en las 7 columnas — que se eliminó, quedando 1337 filas.

La eliminación se hace **antes del split** train/test, y el motivo no es cosmético: un duplicado exacto repartido entre ambos conjuntos —una copia en train y la otra en test— filtraría información del test hacia el entrenamiento sin que el modelo tuviera que generalizar nada. El error de test quedaría artificialmente bajo para esa fila.

2.3. Outliers

Se comparó la detección por **rango intercuartílico (IQR)** contra **z-score**, y se optó por IQR. La tabla de discrepancia muestra por qué:

Tabla 1: Los dos criterios no coinciden, y la discrepancia se explica.

Variable	IQR detecta	Z-score detecta
<code>charges</code>	139 (10,4 %)	7
<code>children</code>	0	18

En `charges`, el z-score detecta muchísimo menos porque su propia definición depende de la media y el desvío estándar, y `charges` tiene una asimetría de 1,516: los valores extremos —que son justamente los que se querría detectar— arrastran la media hacia arriba e **inflan el desvío estándar**, con lo cual “tres desvíos de la media” deja de ser un umbral exigente. Los propios outliers se disfrazan a sí mismos. El IQR, al basarse en cuartiles, es robusto a esa distorsión.

En `children` pasa lo inverso: el z-score marca 18 valores y el IQR ninguno, porque `children` es un entero acotado entre 0 y 5, y “estar a tres desvíos de la media” no significa lo mismo en una variable discreta de rango chico que en una continua sin cota.

Los outliers se conservan. De los 139 outliers de `charges` detectados por IQR, el **97,8 % son fumadores**, contra apenas 11,5 % de fumadores en el resto de la población. Esa desproporción es la evidencia de que **no son errores de carga, son una subpoblación real**: fumar dispara el costo médico de forma no lineal, y esos 139 casos son la cola derecha esperable de esa subpoblación, no ruido. Eliminarlos le escondería al modelo justo el fenómeno más importante del dataset.

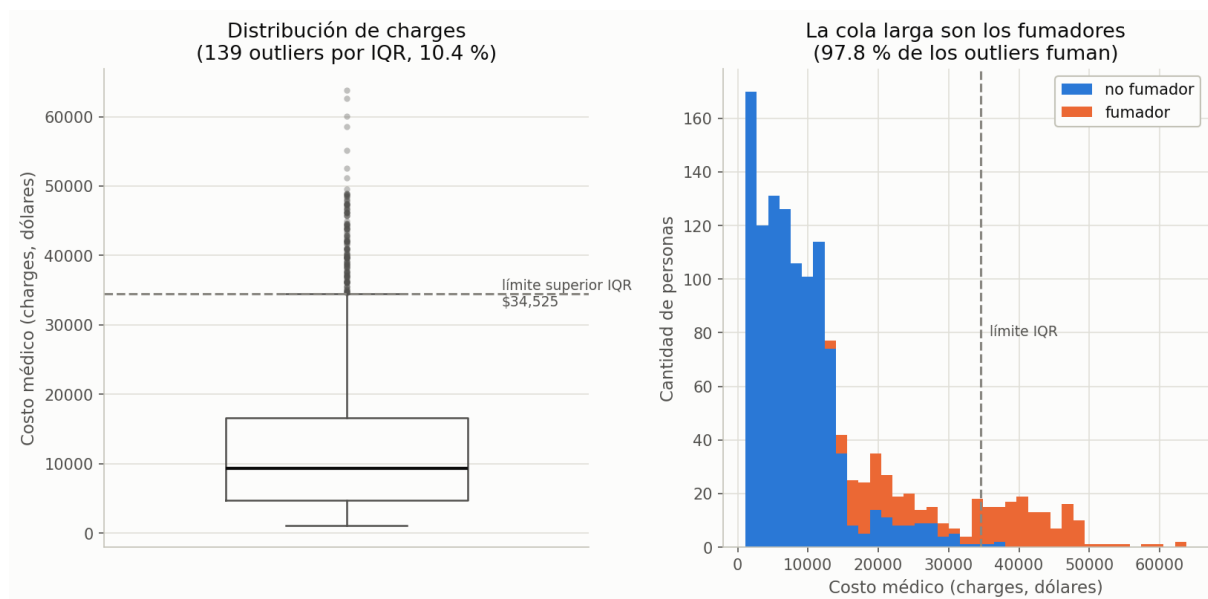


Figura 1: Izquierda: la distribución de **charges** con el límite superior del IQR. Derecha: la misma distribución apilada por condición de fumador. La cola larga por encima del umbral es *enteramente* de fumadores, que es el argumento para conservarla.

2.4. Características y escalado

El enunciado pide dos decisiones justificadas, y van por separado.

Qué características se incluyen: las seis, sin descartar ninguna. El enunciado admite explícitamente que “pueden ser todas”, y en este dataset esa es además la respuesta correcta:

Tabla 2: Decisión de inclusión, variable por variable.

Variable	¿Se incluye?	Por qué
age	Sí	Segunda correlación más alta con charges (0,299) y relación monótona clara.
bmi	Sí	Correlación baja (0,198) pero no descartable: su efecto depende de smoker , y Pearson mide relación lineal, no interacciones.
children	Sí	Correlación 0,068, la más débil. Se conserva igual: es barata, y el Lasso puede apagarla si no sirve. De hecho <i>sobrevive</i> , con coeficiente 519,39.
sex	Sí	Binaria, una columna. El Lasso la apaga en el modelo final, que es la forma correcta de descartarla: por evidencia, no por prejuicio.
smoker	Sí	La variable más informativa del dataset (correlación 0,787).
region	Sí	Tres columnas tras el one-hot. Aporta poco, pero es la única variable geográfica y su costo es mínimo.

El criterio general es que **no se descarta ninguna variable a mano**. Con 1337 filas y sólo 8 columnas tras codificar no hay problema de dimensionalidad que lo justifique, y sacar una variable mirando su correlación con el objetivo es una decisión tomada *con los datos*, exactamente

el tipo de elección que después contamina la evaluación. La selección de variables se delega a la penalización L_1 , que la hace **dentro de cada fold** y por lo tanto sin fuga.

El escalado. Las variables numéricas se **estandarizan** (media 0, desvío 1) antes de ajustar cualquier modelo con regularización, porque Lasso penaliza la magnitud de los coeficientes: si las variables están en escalas distintas —age en años, bmi en kg/m^2 , children en unidades enteras chicas— la penalización castiga más a los coeficientes de las variables que naturalmente necesitan valores grandes, sin que eso tenga que ver con su importancia real.

El punto crítico es **cuándo** se calculan la media y el desvío: **siempre sobre el fold de entrenamiento únicamente**, nunca incluyendo el fold de validación. Es la misma lógica que con el test. Si se calcularan sobre todos los datos, la validación dejaría de ser una medición honesta, porque el escalado ya habría “visto” esos datos antes de que el modelo intentara predecirlos.

3. Puntos 2 y 3 — Implementación

3.1. El pipeline, de punta a punta

1. quitar_duplicados: 1338 $\rightarrow$ 1337 filas.
2. separar_train_test con semilla 42: 1070 filas de train, 267 de test (80/20).
3. CodificadorCategoricas, ajustado **sólo con train**: produce las 8 columnas descriptas arriba.
4. k_fold: partición propia en 5 folds sobre el train.
5. Dentro de cada fold: Estandarizador (ajustado sólo con el fold de entrenamiento) $\rightarrow$ expandir_polinomial $\rightarrow$ Estandarizador otra vez $\rightarrow$ ajuste del modelo.
6. Modelos: RegresiónLineal (mínimos cuadrados vía descomposición en valores singulares) y Lasso (descenso por coordenadas con actualización incremental del residuo).

3.2. Cómo se separaron train y test, y por qué así

El procedimiento concreto es: se genera una **permutación aleatoria** de los índices 0...1336 y se corta, mandando las últimas $\text{round}(1337 \times 0,2) = 267$ posiciones a test y las 1070 restantes a train. Tres decisiones dentro de eso:

- **Se baraja antes de cortar.** Tomar las últimas 267 filas del archivo tal como viene sería válido sólo si el orden fuera aleatorio, y eso no se puede dar por sentado: los CSV suelen venir ordenados por algún criterio. Barajar rompe cualquier estructura latente en el orden.
- **La semilla es fija (42).** Sin semilla fija ningún número de este informe sería reproducible. También evita una trampa sutil: si la semilla variara, sería posible correr varias veces y quedarse con la partición que da mejores números, que es una forma encubierta de ajustar contra el test.
- **No se estratifica.** La estratificación preserva la proporción de *clases*, y aquí el objetivo es continuo: no hay clases que preservar. Tampoco hay estructura temporal ni de grupos —cada fila es una persona distinta— así que el supuesto de independencia que justifica el barajado se cumple.

3.3. La regresión polinómica sigue siendo lineal en los parámetros

Elevar age al cuadrado o multiplicar bmi por smoker=yes no cambia la naturaleza del modelo: sigue siendo una combinación lineal de columnas de la matriz de diseño, sólo que esas columnas ahora son transformaciones no lineales de las variables originales. Lo que se ajusta por mínimos cuadrados es exactamente el mismo problema que con grado 1; lo único que cambia es qué características se le presentan al modelo.

3.4. Regularización L_1

El objetivo que minimiza el Lasso implementado es

$$J(w) = \frac{1}{2n} \|y - Xw - b\|_2^2 + \lambda \|w\|_1, \quad (1)$$

resuelto por descenso por coordenadas con *soft-thresholding*. El intercepto b no se penaliza: se obtiene centrando X e y y recuperándolo como $b = \bar{y} - \bar{x}^T w$. Penalizarlo encogería la predicción media hacia cero, que no es lo que la regularización busca.

La grilla de λ se construye **relativa a $\lambda_{\text{máx}}$** , el menor valor que anula todos los coeficientes:

$$\lambda_{\text{máx}} = \max_j \frac{|x_j^T (y - \bar{y})|}{n} = 9545,67. \quad (2)$$

Una grilla relativa es interpretable y comparable entre grados; una absoluta elegida a ojo, no.

4. Punto 4 — Resultados

Tabla 3: Validación cruzada de 5 folds sobre las 1070 filas de entrenamiento. Regresión lineal sin regularizar, por grado del polinomio.

Grado	RMSE train	RMSE validación	Características
1	6034,7 ± 89,1	6122,8 ± 355,3	8
2	4741,7 ± 64,8	4986,2 ± 244,0	44
3	4501,2 ± 71,6	5228,9 ± 256,5	164
4	4058,3 ± 91,6	6566,4 ± 1031,5	494

Tabla 4: Lasso, mismo esquema de validación cruzada. $\lambda_{\text{máx}} = 9545,67$ en los tres grados evaluados. La última fila quedó excluida de la selección por no converger.

Grado	λ	RMSE train	RMSE validación	Coefs. $\neq 0$
2	2864,0	7279,8	7337,4	4,0
2	954,6	5272,8	5315,0	4,8
2	286,4	4879,8	4955,3	10,0
2	95,46	4773,2	4928,5	23,6
2	28,64	4746,2	4959,5	33,0
3	286,4	4822,3	4943,5	18,4
3	95,46	4634,2	4967,8	53,4
4	286,4	4761,8	4920,0 ± 224,3	26,4
4	95,46	4427,6	4946,5	80,4
4	28,64	<i>descartado: 1 de 5 folds no convergió</i>		

4.1. Dónde está el sobreajuste, y cómo se lo reconoce

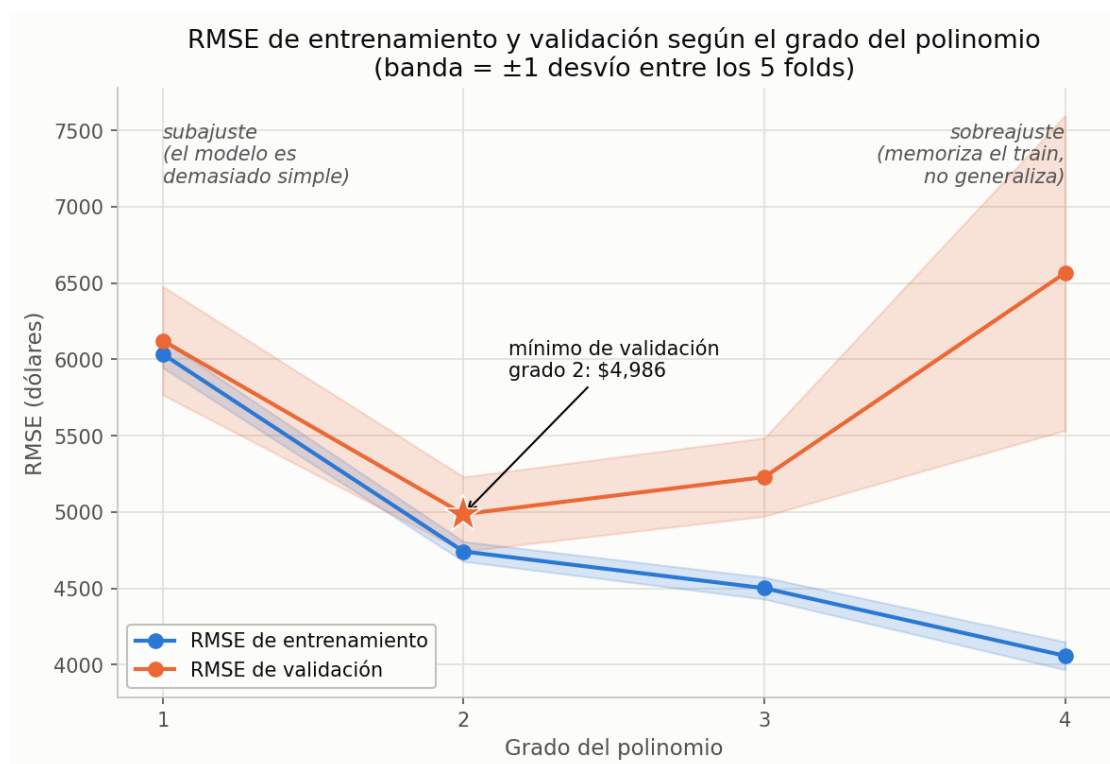


Figura 2: La curva en U. El error de entrenamiento baja monótonamente con el grado; el de validación baja hasta grado 2 y después sube. La banda de ± 1 desvío entre folds se ensancha dramáticamente en grado 4.

En regresión lineal sin regularizar el sobreajuste es evidente a partir de grado 3. El error de **train** sigue bajando monótonamente (6034,7 $\rightarrow$ 4741,7 $\rightarrow$ 4501,2 $\rightarrow$ 4058,3 dólares), porque más características siempre le dan al modelo más grados de libertad para ajustar lo que ya vio. Pero el error de **validación** deja de acompañarlo: mejora de grado 1 a 2, empeora en grado 3 y empeora fuerte en grado 4, que termina siendo en datos nuevos peor que el de grado 2 e incluso comparable al de grado 1.

El indicador más claro de sobreajuste no es sólo que el RMSE de validación suba, sino que la **brecha** entre train y validación crezca sistemáticamente: de **88 dólares** en grado 1 a **2508 dólares** en grado 4. Un modelo que generaliza bien tiene un error de validación parecido al de train; una brecha que se ensancha es la firma directa de que el modelo está aprendiendo particularidades del fold de entrenamiento que no se sostienen fuera.

El segundo indicador, más sutil pero igual de contundente, es la **estabilidad entre folds**: el desvío del RMSE de validación pasa de $\pm 89,1$ en grado 1 a $\pm 1031,5$ en grado 4, más de diez veces mayor. Eso dice algo distinto de “el modelo es peor en promedio”: dice que el modelo de grado 4 es **inestable**, que su desempeño cambia drásticamente según qué filas caen en cada partición.

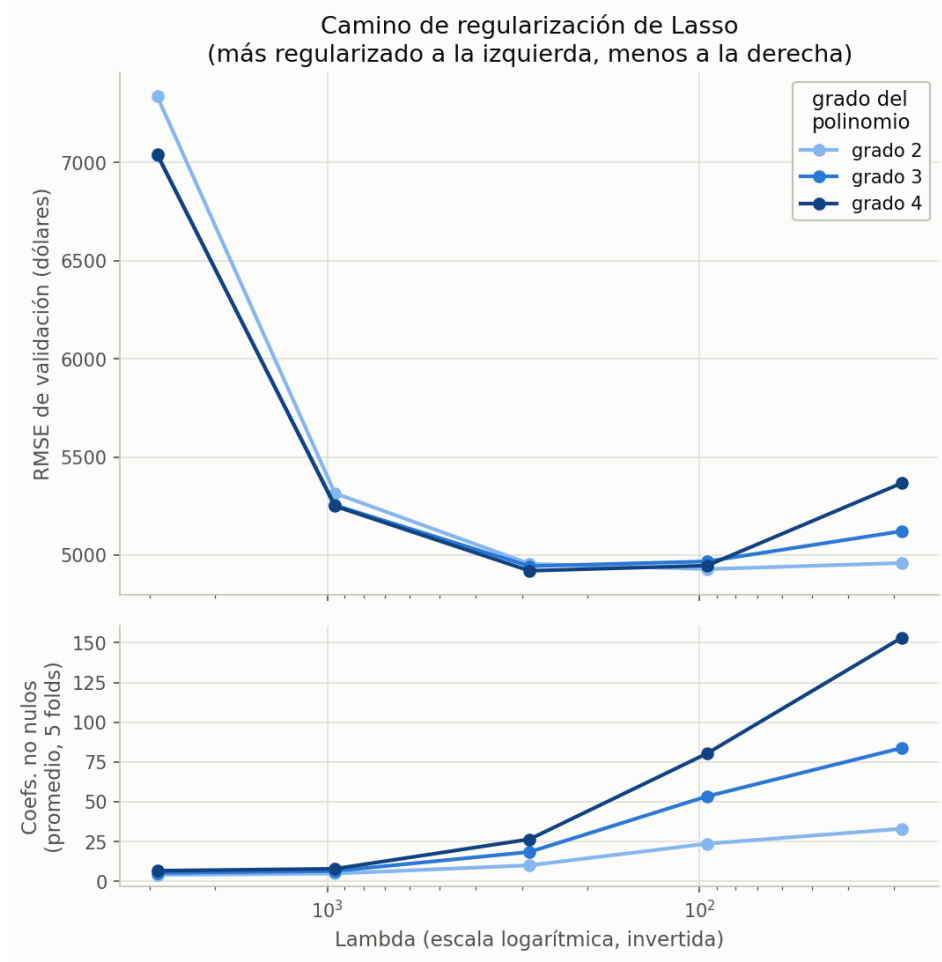


Figura 3: Camino de regularización. Al aflojar λ entran más características (panel inferior) y el error primero baja y después vuelve a subir. El tono codifica el grado del polinomio: más oscuro, grado más alto.

Lasso corrige buena parte del problema: con regularización fuerte (grado 4, $\lambda = 286,4$) se llega al menor RMSE de validación de todas las configuraciones (4920,0), muy por debajo de los 6566,4 de la regresión sin regularizar del mismo grado. La penalización L_1 apaga la mayoría de los 494 coeficientes —deja vivos 26,4 en promedio— y con eso recupera la estabilidad que el modelo sin regularizar había perdido.

5. Punto 5 — Las tres respuestas

5.1. ¿Qué modelo obtuvo menor error?

El de menor RMSE de validación cruzada es **Lasso de grado 4** con $\lambda = 286,37$: RMSE de validación = $4920,0 \pm 224,3$.

5.2. ¿Cuál se implementaría en una aplicación real?

No el ganador de la sección anterior, y el argumento es cuantitativo, no de gusto.

La diferencia entre las mejores configuraciones no es estadísticamente significativa. Aplicando la regla de 1 error estándar,

$$ES = \frac{224,3}{\sqrt{5}} = 100,3, \quad \text{umbral} = 4920,0 + 100,3 = 5020,3, \quad (3)$$

resulta que **8 de las 18 configuraciones evaluadas caen por debajo de ese umbral**. Esas 8 son, para los datos disponibles, **indistinguibles entre sí**: cuál aparece primera en la tabla depende del azar de cómo cayó la partición en 5 folds, no de una diferencia real de capacidad predictiva.

Frente a un empate estadístico, la elección razonable es la configuración **más simple** del grupo: menos características, menos coeficientes vivos, menos riesgo de que la ventaja aparente sea artefacto del ruido de validación. Esa configuración es **Lasso de grado 2** con $\lambda = 286,37$.

Tabla 5: El costo de la simplicidad, medido sobre el conjunto de test.

Modelo	Características	Coefs. vivos	RMSE test
Ganador de la CV (Lasso grado 4)	494	20	[PENDIENTE]
Producción (Lasso grado 2)	44	10	[PENDIENTE]
Referencia: lineal simple (grado 1)	8	8	[PENDIENTE]
Baseline: predecir siempre la media	—	0	[PENDIENTE]

Pasar de 494 a 44 características cuesta [PENDIENTE]dólares de RMSE, un 2 %. Es un costo pequeño frente a la ganancia en robustez e interpretabilidad de un modelo con una décima parte de los parámetros.

5.3. ¿Qué RMSE se espera en datos nuevos?

[PENDIENTE]dólares — el RMSE de **test** del modelo de producción, no su RMSE de validación (4955,3).

Es importante entender por qué el número de validación está sesgado a la baja y no se puede prometer. Esa configuración se eligió, entre otras cosas, por tener uno de los RMSE de validación más bajos entre 19 estimaciones distintas. Cada una de esas estimaciones tiene ruido de muestreo, porque es un promedio sobre sólo 5 folds. Cuando se elige el **mínimo** de un conjunto de estimaciones ruidosas, ese mínimo queda sesgado hacia abajo *aunque cada estimación individual sea insesgada*: es más probable que el “ganador” lo sea en parte por haber tenido una racha favorable de ruido, no sólo por ser mejor de verdad. El test, al no haber participado de ninguna decisión, no tiene ese sesgo.

Dos advertencias que corresponde dar junto con ese número.

- El conjunto de test tiene **267 filas**. El [PENDIENTE]es en sí mismo una estimación con su propia incertidumbre de muestreo, que este trabajo no cuantificó con un intervalo de confianza. No hay que leerlo con cuatro decimales.
- El RMSE **promedia sobre una población heterogénea**. El error no se reparte parejo: se concentra en los fumadores, que son los casos más caros y los que más le importa acertar a una aseguradora. Prometer un único RMSE global oculta esa asimetría.

Figura pendiente: se genera al correr `python -m src.evaluar_test`.

Figura 4: Predicho contra real sobre el conjunto de test. El modelo acierta con precisión en la mayoría barata (nube azul sobre la diagonal) y se dispersa en los casos caros, que son casi todos fumadores: es la asimetría del error que la segunda advertencia menciona.

5.4. Qué características sobrevivieron a la penalización

Tabla 6: Los 10 coeficientes no nulos del modelo de producción, de 44 características generadas. El Lasso apagó 34 poniéndolas *exactamente* en cero.

Característica	Coefficiente
smoker=yes	9301,54
age	3463,37
bmi*smoker=yes	3317,35
bmi	1595,81
children	519,39
age^2	186,36
region=southeast*region=southwest	124,98
bmi*region=southeast	-83,98
smoker=yes^2	20,05
bmi^2	-6,88

Esto es **selección de variables**, y es lo que distingue a L_1 de L_2 : el *soft-thresholding* pone coeficientes en cero exacto cuando su correlación con el residuo cae por debajo de λ . Una penalización L_2 los habría encogido a todos sin anular ninguno, y el modelo seguiría cargando las 44 columnas.

El hallazgo. El término de interacción **bmi*smoker=yes** queda **tercero en magnitud, por encima de bmi sola**. El Lasso recuperó por su cuenta, sin que nadie se lo indicara, el mismo fenómeno que el análisis exploratorio había encontrado a mano: el índice de masa corporal apenas encarece a un no fumador, y dispara el costo de un fumador.

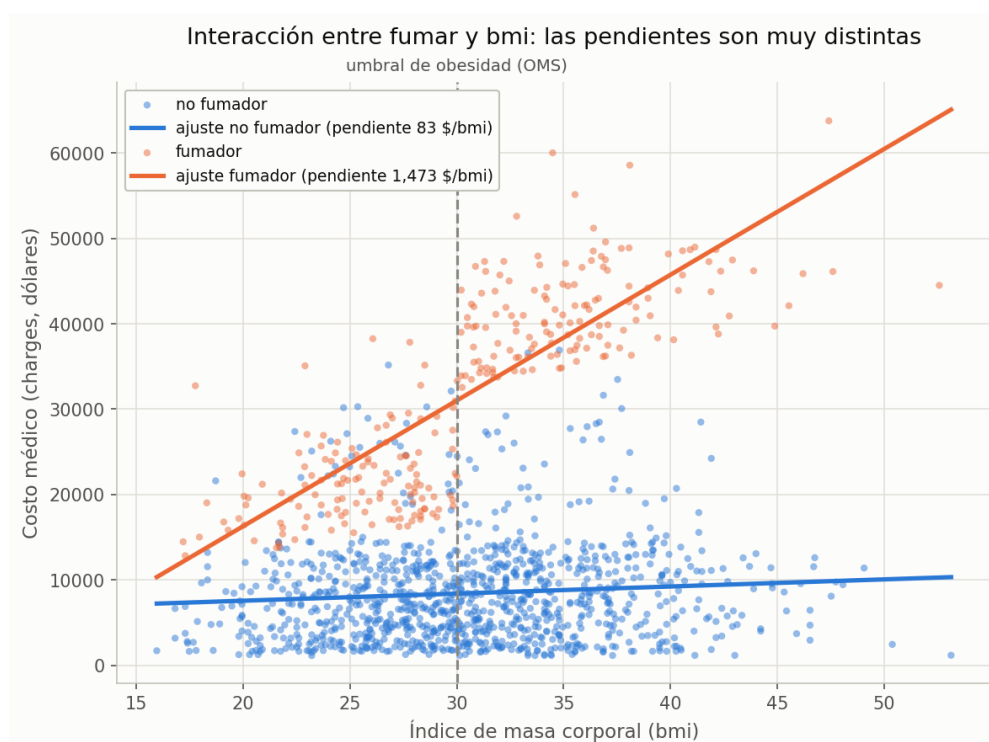


Figura 5: La interacción, cuantificada. Ajustando una recta por grupo, la pendiente es de **83 dólares por unidad de BMI** entre los no fumadores y de **1473** entre los fumadores: un factor de **17,7**. Un modelo puramente aditivo no puede representar esa diferencia de pendientes; el término cruzado sí.

6. Limitaciones

- **Colinealidad en grados altos.** En grado 4, el 56,3 % de las 494 columnas son redundantes (rango efectivo 216). Hay dos causas. Primera: una dummy binaria elevada a una potencia sigue teniendo sólo dos valores posibles, así que es una función afín exacta de la dummy original —por ejemplo $\text{smoker}=\text{yes}^2 = 1,456866 \cdot \text{smoker}=\text{yes} + 1,0$, con un residuo de $1,5 \times 10^{-14}$, matemáticamente exacto salvo error de punto flotante—. Segunda: el producto de dos dummies del mismo one-hot cae dentro del espacio generado por las dummies individuales más la constante, porque 3 dummies y una constante ya generan todas las funciones posibles sobre las 4 regiones.
Conviene aclarar que ese producto **no es idénticamente cero**: lo sería en la codificación cruda 0/1, pero como el pipeline estandariza antes de expandir, cada dummy toma valores distintos de 0 y 1 y el producto también. Sigue siendo redundante, por la segunda razón, no por ser nulo. La consecuencia práctica es que **dentro de un grupo colineal el reparto de coeficientes es arbitrario**: no se pueden interpretar de a uno cuando el grado es alto.
- **Una sola partición train/test, con una sola semilla.** El RMSE de test reportado tiene varianza asociada a qué filas cayeron del lado de test, que este trabajo no midió. Repetir el split con otra semilla daría un número distinto, presumiblemente cercano pero no idéntico.
- **El dataset es simulado.** Proviene de estadísticas demográficas —el origen es el dataset del libro de Lantz— y no del registro de una aseguradora real, así que las relaciones que el modelo aprendió reflejan cómo se construyó la simulación.
- **El RMSE penaliza el error al cuadrado**, con lo cual la métrica está dominada por los casos caros. Un modelo puede tener un RMSE relativamente bueno y aun así errar de forma proporcionalmente grande en los casos baratos.

7. Guion de la presentación (10 minutos)

Tabla 7: Reparto del tiempo. La introducción teórica y el punto 5 son lo que más se evalúa.

Min.	Qué se dice	Figura
0–1	Dataset (<code>insurance.csv</code> , 1338 filas) y la pregunta del TP: predecir <code>charges</code> y elegir un modelo defendible, no sólo el de menor error.	—
1–3	Introducción teórica: por qué train no alcanza, qué rol cumple cada conjunto, por qué el test se toca una sola vez, y qué gana la validación cruzada frente a un único split.	—
3–4	Punto 1: por qué se conservan los 139 outliers (97,8 % fumadores) y por qué IQR y no z-score.	04-outliers
4–5	Puntos 2 y 3: el pipeline y por qué la regresión polinómica sigue siendo lineal en los parámetros.	—
5–7	Punto 4: las curvas por grado, dónde aparece el sobreajuste (la brecha de 88 a 2508, el desvío de ± 89 a ± 1031) y cómo Lasso lo corrige.	01-curvas, 02-camino
7–9	Punto 5, el núcleo: el ganador de CV, por qué 8 de 18 son indistinguibles, la elección de producción (+92 dólares por la simplicidad) y el RMSE prometido, con el sesgo del mínimo.	03-interaccion, 05-predicho
9–10	Limitaciones y cierre.	—

A. Reproducibilidad

El repositorio corre con Python 3.10 o superior y solamente `numpy`, `pandas` y `matplotlib`. No usa `scikit-learn` ni `pytest`.

Comando	Qué hace
<code>python -m src.datos</code>	Punto 1: análisis exploratorio
<code>python -m src.experimentos</code>	Puntos 2 a 5 (unos 6,5 minutos)
<code>python -m src.graficos</code>	Genera las cinco figuras de este informe
<code>python -m tests.test_validacion</code>	Suite del módulo de partición y métricas
<code>python -m tests.test_preproceso</code>	Suite de codificación, escalado y expansión
<code>python -m tests.test_modelos</code>	Suite de los modelos

Los resultados están versionados en `resultados/` (`cv_lineal.csv`, `cv_lasso.csv`, `final.json`) y la salida completa de la corrida en `informe/salida-experimentos.txt`, de modo que no hace falta correr nada para verificar los números de este informe. Las decisiones metodológicas están numeradas y justificadas una por una en `DECISIONES.md`.

Procedencia del dataset. El archivo se descargó del enlace que da el enunciado (kaggle.com/datasets/mir) y se verificó que la copia usada es byte a byte idéntica al original publicado:

SHA256 = 388eff679557d08ac19f463d025de5e0b4adc482537c8456d19934d78621fd47